

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

23/06/2022

Last (family) Name _____

First (given) Name _____

Matricola

Codice Persona

Professor: ☐ Zanero ☐ Carminati ☐ Barenghi

Have you done any challenges/homework, even partially? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading ☐ Yes, Motivation:

[] No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 18 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book". The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct delivery of the exam.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Consider password managers, software applications designed to store and manage online credentials. They also generate passwords. Usually, these passwords are stored in an encrypted database (locally or in the cloud) and locked behind a master password. Entering your master password unlocks your password vault, and from your vault, you can then retrieve whatever password you need.

[2 points] 1. List the advantages and disadvantages of such tools from a security perspective. (the number of rows does not represent the total number of correct answers. If you need more space, continue under the table.)

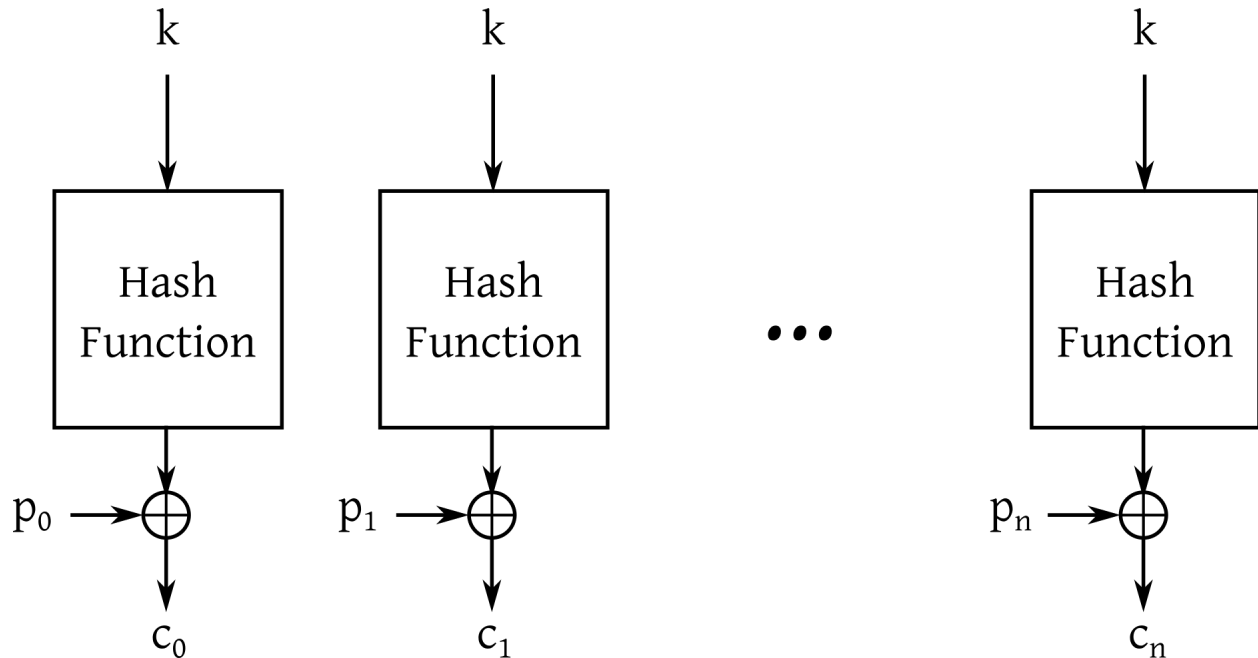
Advantages	Disadvantages
Automatic generation of secure passwords [0.5]	One password [0.5]
Strong, memorable password [0.5]	Additional potentially vulnerable software [0.5]
Autofill (no need to type the password)	Single point of failure

[2 points] 2. Suggest and briefly explain, in rough order of prevalence (i.e., frequency) the two most likely potential attacks against password managers. Do not focus only on the software, but on the whole infrastructure that enables its functioning.

1) Master password guessing/bruteforcing [1]
2) Exploit a vulnerability in the application that implements the master password [1]

Exercise 2 - Introduction to Cryptography (6 points)

Consider the following construction, where k is a 1024 bit long string, known only to Alice and Bob, who want to have confidential communications. The plaintext p is split in $n+1$ blocks: $p_0, p_1, \dots, p_n$. Hash Function is a cryptographic hash function with a 512 bit digest. The ciphertext, which is sent on the line, is formed by the $c_0, c_1, \dots, c_n$ blocks.



[3 points] Is the communication with the depicted scheme (computationally) confidential against a passive attacker, who only eavesdrops on the ciphertext? If yes, argue why; if not, describe concisely what information is leaked, and propose a countermeasure to fix the information leaked.

[2] No. Plaintext blocks with the same value will be encrypted to ciphertext blocks with the same value. Furthermore, if the attacker knows the position of a zero-filled plaintext block, she'll be able to decrypt all the messages sent.

[1] Fix by appending a *nonce* to k , increase the *nonce* for each block.

[3 points] Is the communication with the depicted scheme providing message integrity through the use of the cryptographic hash functions? If yes, argue why; if not, describe concisely a way for the attacker to tamper with the plaintext, and propose a countermeasure to fix the information leaked.

[1.5] No. The hash function is employed as a CSPRNG, it has no other purpose. [1.5] Add a MAC (e.g., CMAC) computed on the ciphertexts.

Exercise 3 - Binary Vulnerabilities (6 points)

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04 #include <stdint.h>
05
06 typedef struct {
07     char buf[36];
08     char *p_msg;
09 } data_t;
10
11 int vuln(char sel){
12     data_t data;
13
14     data.p_msg = data.buf;
15
16     while (sel == 'w' || sel == 'r'){
17         if (sel == 'r'){
18             printf(data.p_msg);
19         } else if (sel == 'w') {
20             gets(data.p_msg);
21         }
22         if (sel == 'x') {
23             system("/bin/sh\0");
24         }
25         scanf("%c", &sel);
26         getchar();
27     }
28 }
29
30 void main(int argc, char** argv){
31     char sel;
32
33     scanf("%c", &sel);
34     getchar();
35     vuln(sel);
36 }
```

Assume that:

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the x86 architecture (32 bit) and for an environment that adopts the usual `cdecl` calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).*
- `getchar()` reads the next character from stream and returns it as an unsigned char cast to an int, or EOF on end of file or error. It is used to correctly manage the `scanf()`: It is not relevant for the exploitation process, you can ignore it for the questions below.

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	Motivate and Modify the line to solve the detected vulnerability
Buffer Overflow	20 [0.25]	See Slides [0.25]
Format String	18 [0.25]	See Slides [0.25]

[2 points] 2. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90.**

2.a) Describe **all the steps and assumptions** required for the successful exploitation of the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, (if any), the input provided during the execution, if multiple executions are necessary.

- **At line 33 'w' must be passed as input**
- **The function vuln will be called passing 'w' as argument.**
- **The branch 'else if (sel == 'w') {' will be executed, enabling the call of the vulnerable line 20 in which the attacker will exploit the vulnerability to overwrite sEIP with addr of buf**

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

Before the vulnerable line			After the vulnerable line		
Low addresses					
^		Function frames			
	buf[0-3]	<- ESP	vuln()	0x90 0x90 0x90 0x90	<-X
	buf[4-7]		vuln()	0x90 0x90 0x90 0x90	
	buf[8-11]		vuln()	0x90 0x90 0x90 0x90	
	buf[12-15]		vuln()	0x90 0x90 0x90 0x90	
	buf[16-19]		vuln()	0x20 0x30 0x40 0x50	
	buf[20-23]		vuln()	0x60 0x70 0x80 0x90	
	buf[24-27]		vuln()	not_relevant	
	buf[28-31]		vuln()	not_relevant	
	buf[32-35]		vuln()	not_relevant	
	*p_msg		vuln()	not_relevant	
	sEBP	<- EBP	vuln()	not_relevant	
	sEIP		main()	~X	
	sel (w)		main()	not_relevant	
	sel (w)		main()	not_relevant	
	sEBP		main()	not_relevant	
	sEIP			not_relevant	
	argc			not_relevant	
	argv			not_relevant	
High addresses					

[1 point] 3. **Focus only on the stack-based buffer overflow(s) you found.** Consider ONLY the correctly implemented “Stack Canary” mitigation technique (i.e, randomly generated and non-guessable). Is it effective to prevent your exploit at point 2. ?

If the mitigation technique is effective, explain why and describe how you would modify the exploit to bypass the mitigation in order to **execute the same shellcode of the previous question.**

If it is not effective, please explain why.

Yes, it is effective ... [see slides] [0.25]

[0.75] In order to exploit the aforementioned limitation it is necessary to perform 3 iterations of the while cycle.

1. In the first run, it is necessary to insert ‘w’ at line 33, in order to exploit the BO at line 20 and overwrite *p_msg with the address of canary, which can be easily retrieved since no ASLR is enabled.
2. At line 25, it is necessary to insert ‘r’, to call the printf at line 18 and print the current value of the canary
3. At line 25, it is necessary to insert ‘w’, and exploit the BO at line 20 to write in the buffer “NOPSled+shellcode+canary(retrieved in the previous step)+address_of_NOPSLED”

[2 points] 4. A friend of yours suggests that there is a way **to exploit the vulnerability/ies you found** to spawn the privileged shell at line 23 in the program **without executing any shellcode or return oriented programming (ROP) technique.**

If it is possible, please explain **all the steps and assumptions** required for a successful exploitation of the vulnerabilities. **Consider one vulnerability at a time.** Include also any assumptions on how you must call and execute the program.

If it is not effective, please explain why.

Buffer overflow ONLY:

[1] It is necessary to insert 'w' at line 33, in order to exploit the BO at line 20 and overwrite sel with 'x' to execute line 22.

OR

1. 'w' -> *sel
2. 'w' -> sel='x'
3. Line 22 is executed

Format string only ONLY:

[1]

In order to exploit the aforementioned vulnerability it is necessary to perform 2 iterations of the while cycle:

1. It is necessary to insert 'w' at line 33, in order to exploit the BO at line 20 and write into the buffer the format string exploit (to write sel=x)
2. At line 25, it is necessary to insert 'r', to call the vulnerable printf.

Exercise 4 - Web Vulnerabilities (6 points)

You have been hired by the Los Santos Regional Laboratory and you have been asked to carry out penetration testing on the laboratory's infrastructure. They have set up a mock environment for you to experiment on. Your test will be limited to a portion of their infrastructure: the retrieval of documents. Documents have different levels of clearance and only users with equal or greater clearance should be allowed to consult them.

The relevant pseudo-code of the infrastructure is the following:

```
10 ...
11 csrf = CSRFProtect()
12 csrf.init_app(app)
13 ...
14
15 # https://lsr1.gov/register
16 def register():
17     if request.method != "POST":
18         abort(403)
19     username = request.form['username']
20     password = request.form['password']
21
22     if not is_alphanumeric_plus(username) or not is_alphanumeric_plus(password):
23         abort(400)
24
25     q = "INSERT INTO User (username, password) VALUES ('" + username + "', '" + password +
26         "');"
27     dbquery(q)
28     q = "INSERT INTO Permissions (user_id, clearance) VALUES ( (SELECT id FROM User WHERE
29         username='" + username + "'" ), 0);"
30     dbquery(q)
31
32     return login_page()
33
34 # https://lsr1.gov/login
35 def login():
```



```
34     if request.method != "POST":
35         abort(403)
36     username = request.form['username']
37     password = request.form['password']
38
39     if not is_alphanum_plus(username) or not is_alphanum_plus(password):
40         abort(400)
41
42     q = "SELECT id, clearance FROM User JOIN Permission WHERE username='" + username + "'
and password='" + password + "'"
43     result = dbquery(q)
44     if result.empty():
45         return login_page(error="Wrong username or password.")
46
47     session['id'] = result['id']
48     session['clearance'] = result['clearance']
49
50     return login_page(id=id)
51
52 # https://lsrl.gov/ins_doc
53 def ins_doc(id):
54     if request.method != "POST":
55         abort(403)
56     user_id = session['id']
57     content = request.form['content']
58
59     q = """INSERT INTO Documents (author_id, content, clearance) VALUES ((?), (?),
60 (SELECT clearance FROM Permissions WHERE id=?))"""
61     result = dbquery(q, user_id, content, user_id)
62
63     return login_page(id=id)
64
65 # https://lsrl.gov/get_doc
66 def get_doc(id):
67     if request.method != "GET":
68         abort(403)
69
70     user_id = session['id']
71     doc_id = request.args.get('doc_id')
72
73     q = """SELECT content, clearance FROM Documents
74 WHERE clearance<=(SELECT clearance from Permissions
75 WHERE id=""" + user_id + """)
76 AND doc_id=""" + doc_id + """"
77     result = dbquery(q)
78
79     html_page = """< ... """ ## abbreviated for brevity
80     if result.empty():
81         html_page += """No documents to display"""
82     else:
83         html_page += result['content']
84     html_page += """... >""" ## abbreviated for brevity
85
86     return html_page
87
```

The website uses a relational database as a back-end. The relevant database info can be inferred from the following sample table, as well as from the code:

User		
id	username	password
1	polino	...
2	ikiga1	...
3	law	...
212	w4rio	...

Documents			
author_id	doc_id	content	clearance
213	D121	I hated those challs!!	0
212	D122	...	2
2	D152	...	5

Permissions	
id	clearance
2	5
3	5
213	0
...	...

Assume the following:

- The function **is_alphanum_plus()** returns true only if the input string contains only letters, numbers, the whitespace, a comma, a full stop, and a question mark e.g., **is_alphanum_plus**("Hey, Jim. Shall we play 3 songs rather than 2?") returns true.
- The class **CSRFProtect**, as used in lines 10-11, correctly implements csrf tokens on every POST, PUT, PATCH, and DELETE methods.
- The id in User is auto-incremental, so it is in Documents.
- The DBMS makes sure that usernames are unique and reject INSERT queries otherwise.
- The function **dbquery** sends a query to the database and returns the response. If called with multiple parameters, dbquery can make a prepared statement: placeholders are denoted by '?' and arguments are taken in order from left to right; if the number of placeholders does not match the number of arguments the query is not executed and the result is empty. To check if the response is empty one can call the method **empty** on the query response object. The result of a query can be accessed as a dictionary (e.g., see line 83).
- The cookie session is correctly managed (i.e., it is not possible to forge arbitrary cookies).
- The library used to access the DBMS does not support stacked queries, i.e., the DBMS does not execute multiple queries if asked on the same query and separated by a semicolon. For instance, the query "select x from y where a=b; update k set [...]" would trigger an error and not be executed.

1. [3 points] Based on the available pseudo-code and assumptions, fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding **line/s** of code and relevant data flow/arguments.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
----------------------------	---	--

Sql Injection (SQL-I)	<i>[0.5]Yes... Line 75-77, doc_id can be arbitrarily crafted...</i>	<i>[0.5]See slides.</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>[0.5]Yes... lines 59-61 the document is inserted with no extra checks. Line 83 is displayed with no html sanitization. Stored XSS.</i>	<i>[0.5] See slides.</i>
Cross-site request forgery (CSRF)	<i>[1]See assumption: The class CSRFProtect, as used in lines 10-11, correctly implements csrf tokens on every POST, PUT, PATCH, and DELETE method.</i>	<i>N.A.</i>

2. [1.5 points] Assume you control a user with clearance 0. Can you leak the content of the classified document D152, whose required clearance is 5? Discuss your exploit in detail, writing any relevant code.

*[0.5] SQL injection in the document retrieval function, lines 73-76.
You need to be logged in etc...*

[1] https://lsrl.gov/get_doc?doc_id=' OR doc_id='D152'; -

3. [1.5 points] Can you leak the password of user 'law'? Discuss your exploit in detail and why it works. State all the relevant assumptions and code.

[0.5] SQL injection in the document retrieval function, lines 73-76.

You need to be logged in etc...

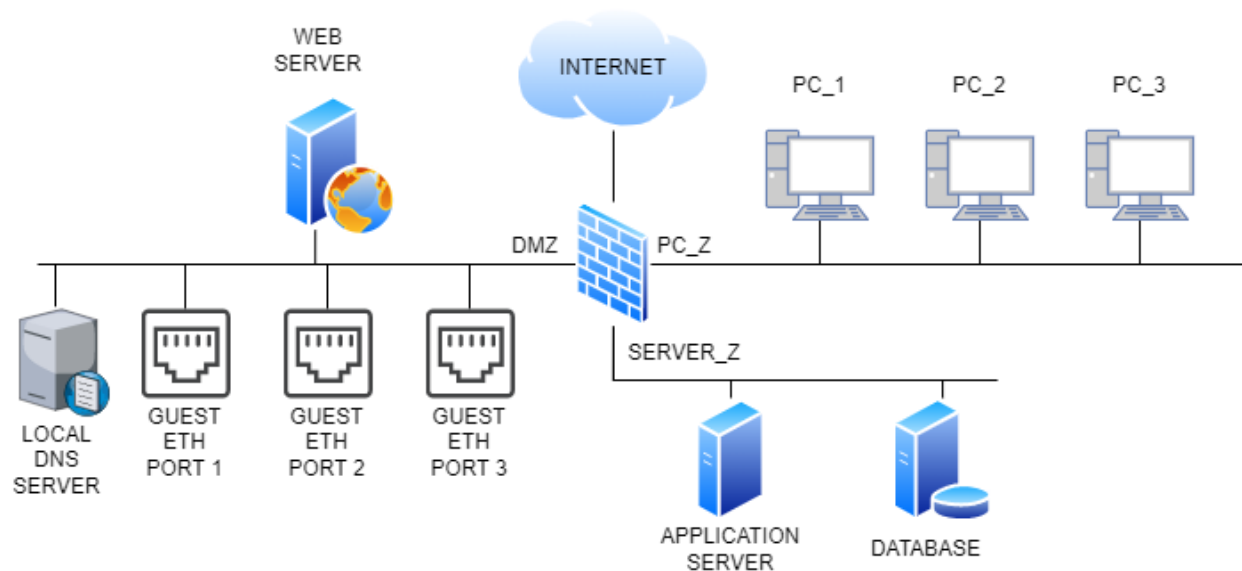
[1] Exploit:

`https://lsrl.gov/get_doc?doc_id=' UNION password, 5 FROM Users WHERE username='law'; -`

The password needs to be aligned with 'content' and the query must be padded with an integer that matches the clearance parameter (to make the UNION work properly).

Exercise 5 - Network Security (6 points)

Karl is the new IT guy in a branch of a medium sized company. Due to the increase of smart working employees, the company has asked Karl to set up a VPN for their employees, so that they can work from home, as if they were **at their desk**. In the figure you can see the current status of the company's internal network, where there is a stateful packet filter firewall, and below you can find the list of firewall rules.



Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
All	All	All	All	All	Deny	Default Deny
PC_X IPs	80/443	PC_Z -> Internet	*	80/443	Allow	Internet communication for all PCs
GUEST_ETH_X IPs	80/443	PC_Z -> Internet	*	80/443	Allow	Internet communication for all Guest users
PC_X IPs	AS Port	PC_Z -> SERVER_Z	AS_IP	AS Port	Allow	Workers access to the application server
AS_IP	DB Port	SERVER_Z -> SERVER_Z	DATABASE_IP	DB Port	Allow	Allow communication between Application server and database
WEB_SERVER	AS Port	DMZ -> SERVER_Z	AS_IP	AS Port	Allow	Web Server access to the application server
PC_X IPs	53	PC_Z -> DMZ	DNS SERVER IP	53	Allow	Local dns server access for workers

5.1 [1.5 point] No one told Karl whether the VPN should be full or split tunneling. Knowing that a lot of workers prefer smart working, and the branch local network already has issues with the internet traffic load, which one should Karl choose? Would the answer change knowing that the company laptops with which the users work from home contain proprietary data and some company's IPs? Justify your response

Given that the network already has too much traffic load, Karl should probably opt for the less network-demanding split tunneling option, which redirects only part of the traffic to the company local network.

However, if the company is particularly worried about the integrity of the traffic that reaches the computers also in case of smart working, they could opt to redirect all the traffic to the local network in order to filter it.

5.1 [2 points] What should Karl do to implement the VPN on the already existing company's setup? You can draw on the image if needed.

Karl should add a VPN Server in the PC_Z of the local network, set up all the laptops that should be allowed to contact it, and update the firewall rules to enable smart workers to contact the VPN Server. [+ FIREWALL RULE]

5.1 [1 point] While working on the VPN setup, Karl realized that there is something wrong with the firewall filters of the company. Can you spot it (and eventually fix it)? Justify your response.

- + All | 80/443 | Internet->DMZ | WEB_SERVER | 80/443 | Allow: Add access to WS from remote
- AS to DB rule is useless: they are in the same network already
- Source port for 1-6 rules should be either Any or 1025-65535
- Local DNS lacks access to the internet on port 53 to perform recursive querying
- The direction of the first packet in the third rule is DMZ -> Internet

5.1 [1.5 point] The local DNS listens on the standard 53 port. Assuming that an attacker obtains access to a guest outlet in the DMZ, would he/she be able to tamper with the local DNS communication? How? Justify your answer.

DNS uses UDP to communicate. Therefore an attacker could exploit the lack of a session in UDP to attempt to answer the local PCs contacting the DNS. However, this still requires the attacker to guess the ID of the DNS exchange, unless he/she is also capable of sniffing the messages passing on the DMZ.

Exercise 6 - Malware Security (6 points)

Our systems have been compromised by a very powerful malware that encrypted all the files in the **/home** directories of our systems. Luckily, we succeeded in collecting two samples. Luckily, we succeeded in collecting two versions of the same malware. The code of the two malware samples is reported below.

Sample 1	
.text	
0x08048047<evil>:	
0x08048047:	push ebp
0x08048049:	mov ebp,esp
0x08048050:	mov eax,0x8090000
0x08048055:	mov ebx,0x8048086
0x0804805a:	mov cl,BYTE PTR [eax]
0x0804806c:	mov dl,BYTE PTR [ebx]
0x0804806e:	cmp cl,0x0
0x08048071:	je 0x08048086
0x08048077:	xor dl,cl
0x08048079:	mov BYTE PTR [ebx],dl
0x0804807b:	add eax,0x1
0x0804807e:	add ebx,0x1
0x08048081:	jmp 0x0804805a
0x08048086:	sub DWORD PTR [ebx+0x2e],eax
0x08048089:	push DWORD PTR [edi]
0x0804808b:	sub ebx,DWORD PTR [eax-0x3f66bf99]
0x08048091:	retf
0x08048092:	mov bh,0xdf
0x08048094:	sub ebp,DWORD PTR [ebx+0x3ef5045b]
0x0804809a:	imul ecx,DWORD PTR [eax+0x40],0x31
0x0804809e:	and al,0x39
0x080480a0:	pop eax
0x080480a1:	mov edi,0x5d97e942
0x080480a6:	loope 0xffffffffb2
0x080480a8:	cli
0x080480a9:	(bad)
0x080480aa:	(bad)
0x080480ab:	movs DWORD PTR es:[edi],DWORD PTR ds:[esi]
0x080480ac:	shl BYTE PTR [ecx],1
0x080480ae:	leave
0x080480af:	ret
.rodata	
0x08090000: <Data with length 40, null terminated>	

Sample 2

.text

```
0x08048046<evil>:
0x08048046:    push    ebp
0x08048048:    mov     ebp,esp

0x0804804a:    mov     ebx,0x0804806b
0x0804804f:    xor     eax,eax
0x08048051:    cmp     eax,0x28
0x08048054:    jge     0x0804806b
0x0804805a:    mov     cl,BYTE PTR [ebx + eax]
0x0804805d:    sub     cl,0x3
0x08048060:    mov     BYTE PTR [ebx + eax],cl
0x08048063:    add     eax,1
0x08048066:    jmp     0x08048051

0x0804806b:    fstp    QWORD PTR [eax+eiz*2+0x22e70b6d]
0x08048072:    jg      0xfffffffffa
0x08048074:    push    ebp
0x08048075:    lea     ecx,ds:0xf130af40
0x0804807b:    fisub   WORD PTR [edx-0x703c814d]
0x08048081:    mov     eax,edi
0x08048083:    or      eax,DWORD PTR ds:0xd19fe443
0x08048089:    bswap   esp
0x0804808b:    stos    DWORD PTR es:[edi],eax
0x0804808c:    sbb     eax,DWORD PTR [eax+0x50]
0x0804808f:    fimul   WORD PTR [esp+edi*4]
0x08048092:    (bad)

0x08048093:    leave
0x08048094:    ret
```

4.1 [2 point] It is clear that the malware is showing evasive behavior. What technique is implemented? Explain how it is implemented in this specific case.

If this malware is metamorphic, explain which instructions implement the obfuscation functionality.

If this malware is evasive, explain how it is evading the environment.

If this malware is polymorphic, describe how the decryption routine is implemented.

[1 tech + 1 motivation] Polymorphism, the two malware samples employ an OTP (1st sample) and a shift-cipher (2nd sample) with $K = 3$ (Caesar's Cipher) to decrypt/unpack the malicious code.

Can't say if the malware is evasive or not since some part of the malware is packed.

4.2 [2 point] We executed the two samples in a Virtual Environment. Tracing the system calls of two different executions with `strace` yields the following outputs (trimmed for readability):

```
Strace 1
...
open("/home/law/.bashrc", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=3979, ...}) = 0
read(3, 0x7f508000, 3979) = 3979
close(3) = 0
open("/home/law/.bashrc", O_WRONLY|O_TRUNC) = 3
write(3, 0x7f508000, 3984) = 3984
close(3) = 0
...
(similar lines as above)
...
exit(0)
```

```
Strace 2
...
sleep(0x1000)
exit(0)
```

[1 point] What kind of technique does the malware employ?

[1 point] Is such a technique adopted by both samples? How can you spot it in both cases?

The malware employs an evasive technique called dormant code. Since it detects that it is running in a VM, it doesn't show its malicious behavior. Such a technique is performed with a sleep of 0x1000 seconds.

Only the second sample employs dormant code by performing a sleep syscall, clearly visible in the strace of its execution.

4.3 [2 points] Is it possible to analyze this malware through static analysis? Why? Which instructions would you consider for fingerprint based detection? Do you expect this would be effective?

No, because we only see the decryption routines.

No relevant instructions can be considered for a fingerprint based detection since the ones of the decryption routines are too generic and would lead to many false positives.

